

App Dev II Project Report

Placement Portal Application V2 (PPA-V2)

1. Student Details

Name: Kshitiz Singh

Roll Number: 25f1001872

Email: 25f1001872@ds.study.iitm.ac.in

About Me: I am a student at IIT Madras BS Degree program with a strong interest in full-stack web application development. I enjoy building practical, real-world applications that solve institutional problems and demonstrate modern software engineering principles.

2. Project Details

Project Title: Placement Portal Application V2 (PPA-V2)

Problem Statement:

To design and build a modern, role-based web application that digitizes and streamlines the campus recruitment process — replacing manual spreadsheets and email chains with a structured, centralized platform for students, companies, and administrators.

Approach:

The application was built as a full-stack SPA (Single Page Application) with a RESTful Flask backend and a Vue.js frontend. The system was migrated and upgraded from a previously built MAD-I Flask/Jinja2 application into a modern decoupled architecture. Redis was integrated for caching and as a message broker for Celery background jobs. JWT-based authentication was implemented for secure, stateless role-based access control across three user roles: Admin, Company, and Student.

3. AI/LLM Declaration

I used Claude (Anthropic) to assist in architectural suggestions, identifying bug locations, generating targeted code snippets for boilerplate sections, and providing guidance on Flask-JWT-Extended configuration.

The extent of AI/LLM usage is around 30-40%, limited to code suggestions, debugging hints, and documentation guidance. All final implementation logic, business decisions, debugging, and integration were done manually by me.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend REST API framework
Vue.js (CDN + Vite)	Single Page Application frontend framework
SQLite + SQLAlchemy	Database and ORM for data persistence
Bootstrap 5	Frontend styling and responsive UI components
Flask-JWT-Extended	JWT-based stateless authentication and RBAC
Flask-CORS	Cross-origin request handling between Vue and Flask
Redis (Memurai on Windows)	Caching layer and Celery message broker
Celery	Asynchronous background job processing
Celery Beat	Periodic task scheduler for reminders and reports
Flask-Mail	Email notifications for reminders and reports
Axios	HTTP client for Vue to Flask API communication
Vue Router	Client-side routing and navigation guards

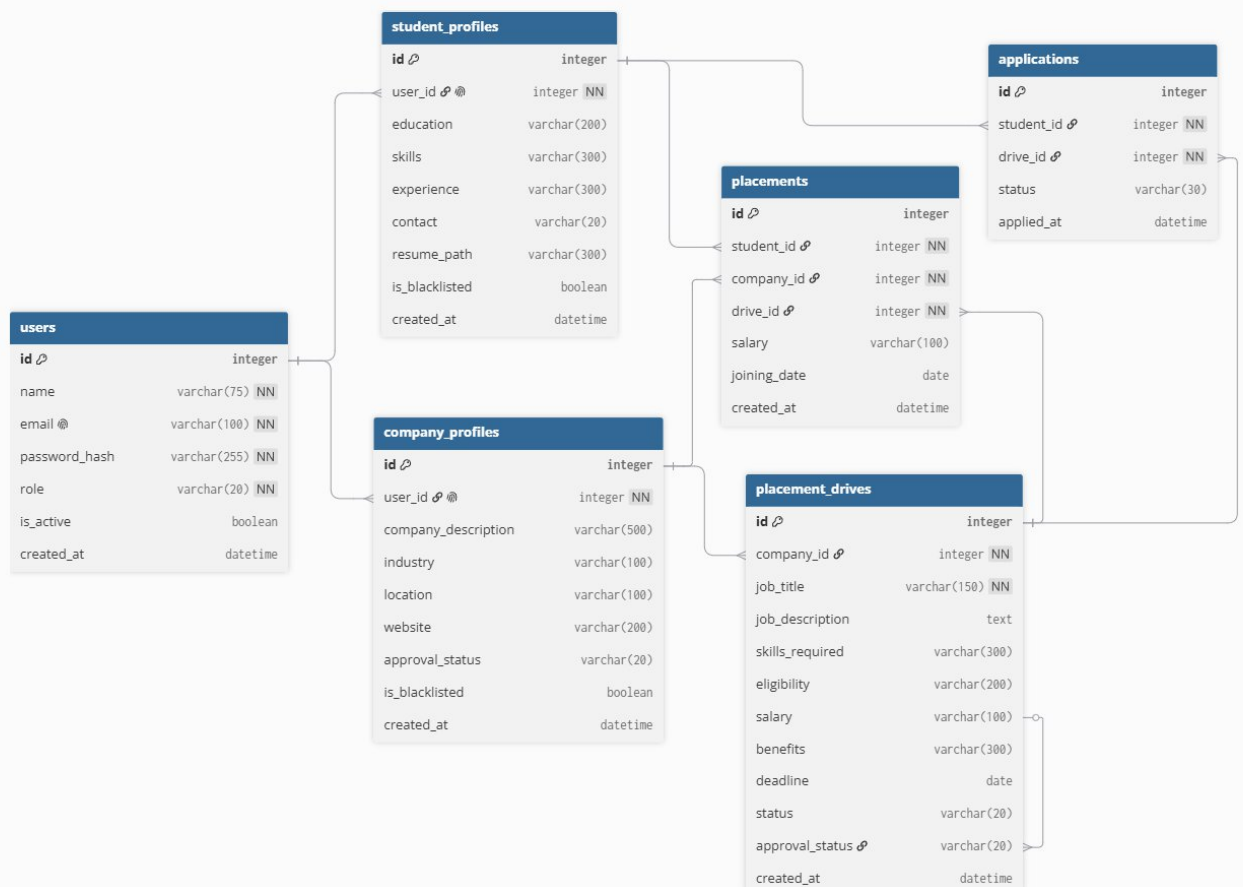
5. Database Schema / ER Diagram

Tables:

- **User** — id, name, email, password_hash, role (admin/company/student), is_active, created_at
- **StudentProfile** — id, user_id (FK), education, skills, experience, contact, resume_path, is_blacklisted, created_at
- **CompanyProfile** — id, user_id (FK), company_description, industry, location, website, approval_status, is_blacklisted, created_at
- **PlacementDrive** — id, company_id (FK), job_title, job_description, skills_required, eligibility, salary, benefits, deadline, status, approval_status, created_at
- **Application** — id, student_id (FK), drive_id (FK), status (Applied/Shortlisted/Interview/Offer/Rejected/Placed), applied_at
- **Placement** — id, student_id (FK), company_id (FK), drive_id (FK), salary, joining_date, created_at

Relationships:

- One-to-One → User → StudentProfile / CompanyProfile
- One-to-Many → CompanyProfile → PlacementDrive
- One-to-Many → StudentProfile → Application
- One-to-Many → PlacementDrive → Application
- One-to-Many → Application → Placement (created when status = Placed)
- Unique Constraint on (student_id, drive_id) in Application — prevents duplicate applications



6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/login	POST	Login for all roles, returns JWT token
/api/auth/register/student	POST	Student self-registration
/api/auth/register/company	POST	Company registration (pending approval)
/api/admin/dashboard	GET	Admin stats — total students, companies, drives, applications
/api/admin/companies	GET	List all companies with search by name/industry
/api/admin/companies/:id/approve	POST	Approve company registration
/api/admin/companies/:id/blacklist	POST	Toggle company blacklist status
/api/admin/drives/:id/approve	POST	Approve placement drive
/api/admin/students/:id/blacklist	POST	Toggle student blacklist status
/api/company/dashboard	GET	Company dashboard with upcoming/closed drives
/api/company/drives	POST	Create new placement drive
/api/company/drives/:id/applications	GET	View all applications for a drive
/api/company/applications/:id/status	POST	Update application status
/api/student/dashboard	GET	Student dashboard with companies and applied drives
/api/student/drives	GET	Browse approved active drives with search
/api/student/drives/:id/apply	POST	Apply to a placement drive
/api/student/applications	GET	View complete application history
/api/student/profile	PUT	Update student profile
/api/tasks/send-reminders	POST	Trigger interview reminder emails (Admin)
/api/tasks/monthly-report	POST	Trigger monthly placement report emails (Admin)
/api/tasks/export/student	GET	Export student applications as CSV
/api/tasks/export/company	GET	Export company applications as CSV

7. Architecture and Features

Architecture Overview:

The application follows a decoupled client-server architecture. The Flask backend exposes a REST API consumed by the Vue.js SPA frontend via Axios HTTP calls. JWT tokens handle authentication without server-side sessions.

Backend Structure (Flask):

- `run.py` — application entry point
- `app/__init__.py` — app factory, initializes db, jwt, mail, celery, registers blueprints
- `app/extensions.py` — db, jwt, mail, celery instances (avoids circular imports)
- `app/config.py` — configuration for DB, JWT, Redis, Mail
- `app/models/` — SQLAlchemy models: User, StudentProfile, CompanyProfile, PlacementDrive, Application, Placement
- `app/routes/` — Flask Blueprints: `auth_routes`, `admin_routes`, `company_routes`, `student_routes`, `tasks_routes`
- `app/tasks/jobs.py` — Celery tasks: `send_interview_reminders`, `generate_monthly_report`, `export_student_csv`, `export_company_csv`
- `app/utils/cache.py` — Redis cache helpers: `get_cache`, `set_cache`, `delete_cache`, `delete_pattern`
- `celery_worker.py` — Celery worker entry point

Frontend Structure (Vue.js):

- `src/main.js` — Vue app entry, mounts router
- `src/App.vue` — root component with RouterView
- `src/router/index.js` — routes with auth guards (requiresAuth, role-based)

- src/utils/api.js — Axios instance with JWT Bearer token interceptor
- src/views/ — page components: HomePage, LoginPage, RegisterStudent, RegisterCompany
- src/views/admin/ — AdminDashboard.vue
- src/views/company/ — CompanyDashboard.vue, CreateDrive.vue, DriveApplications.vue
- src/views/student/ — StudentDashboard.vue

Implemented Features:

- JWT-based authentication with role-based access control (Admin, Company, Student)
- Admin dashboard: stats, company approval/rejection, drive approval, blacklisting, search
- Company dashboard: create drives, view applicants, update application status through full pipeline
- Student dashboard: browse and search drives, apply, track application status, update profile
- Full application status pipeline: Applied → Shortlisted → Interview → Offer → Rejected → Placed
- Automatic Placement record creation when application status is set to Placed
- Duplicate application prevention via DB-level UniqueConstraint
- Redis caching on high-traffic endpoints with 5-minute TTL and cache invalidation on mutations
- Celery + Redis background jobs: interview reminders, monthly reports, CSV exports
- Celery Beat scheduling: daily reminders at 9am, monthly reports on 1st of each month
- Email notifications via Flask-Mail + Gmail SMTP
- Asynchronous CSV export for students and companies

8. Milestones Completed

#	Milestone	Git Commit Message
M0	GitHub Repository Setup	Milestone-0 PPA-V2 Setup
M1	Database Models and Schema Setup	Milestone-1 Database Models and Schema Setup
M2	Authentication and Role-Based Access	Milestone-2 Authentication and Role-Based Access JWT Vue
M3	Admin Dashboard and Management	Milestone-3 Admin Dashboard and Management Flask Vue
M4	Company Dashboard and Job/Application Management	Milestone-4 Company Dashboard Job Application Management Flask Vue
M5	Student Dashboard and Job Application System	Milestone-5 Student Dashboard Job Application System Flask Vue
M6	Job Application History and Status Tracking	Milestone-6 Application History Status Tracking Flask Vue
M7	Backend Jobs — Celery + Redis	Milestone-7 Celery Redis Background Jobs Interview Reminders CSV Export
M8	API Performance Optimization and Caching	Milestone-8 Redis Caching API Performance Optimization

9. Challenges Faced

- JWT Identity Format: Flask-JWT-Extended v4.x requires identity to be a string or int, not a dict. Fixed by using create_access_token(identity=str(user.id), additional_claims={"role": user.role}) and reading role via get_jwt() instead of get_jwt_identity().
- Circular Imports: Solved by creating extensions.py to hold db, jwt, mail, celery instances separately, imported into __init__.py and routes independently.
- Celery on Windows: Celery requires --pool=solo flag on Windows to avoid multiprocessing issues.
- Redis Cache Invalidation: Implemented delete_pattern() to clear stale cache keys when company approval status or drive data changes.
- CORS: Flask-CORS configured to allow Vue dev server (localhost:5173) to communicate with Flask (localhost:5000).

- Schema Migration from MAD-I: Dropped denormalized FK columns (student_name, company_name in Application/Drive tables), expanded Application status from 3 to 6 states, added approval_status to PlacementDrive.

10. Video Presentation

Drive Link: [LINK](#)